

Week 2: Building Vision Pipeline - Datasets, MediaPipe and Spatial Extraction.

Week 2 Introduction and Objectives: -

Having finally built the architectural infrastructure of this Continuous American Sign Language (CSLT) translator over the course of the first week, it was time to move on to more tangible matters, namely the actual implementation. This Week 2 focused on the main goal of creating the base of the data pipeline, which is one of the four stages of the architecture of the proposed project.

To read high-quality, structured spatial data, the Long Short-Term Memory (LSTM) network needs high-quality data in order to analyze the temporal flow of a sign. Consequently, my objectives during the week were to find a robust dataset, build the Python and computer vision environment, and complete the implementation of Google MediaPipe framework to read and normalize 3D spatial landmarks using data streams of raw video feeds.

Obtaining the Data: The WLASW Dataset: -

The quality and variety of the data on which machine learning models are trained is a major limitation to these models. Neural network needs to be shown a broad range of signers, lighting and camera angles in order to reduce bias and overfit.

To perform this initial stage, I merged with Word-Level American Sign Language (WLASL) dataset. Nowadays WLASW is among the largest publicly-available databases of ASL, with thousands of video examples of various, native ASL-signers. Although my final goal is the ability to translate continuous sentences, by using the WLASL videos, I will be able to apply the spatial feature extraction pipeline on clean, annotated gestures and only then merge the challenges of continuous, unsigned signing (which I will discuss later with the help of CTC Loss).

Environmental Preparation of the Vision: -

The video data was secured, and the local development environment was created using Python and OpenCV (Open Source Computer Vision Library). OpenCV is used as the industry standard of real-time video processing. The initial scripts that I wrote were needed to consume the WLASL MP4 files, break them down frame by frame and present them to be processed.

Nonetheless, OpenCV is essentially a mere decoder of pixels and colors. I required a tracking framework that relied on machine-computed learning to extract meaningful anatomical and geometry data necessary to be used in sign language recognition.

An implementation of MediaPipe Holistic: -

In order to follow the complicated physical motions within the ASL, I incorporated Google MediaPipe Holistic model in the OpenCV pipeline. MediaPipe is highly-optimized, which enables the execution of the complex inference models in real-time without a huge enterprise-sized cluster of GPUs.

I chose the pipeline of Holistic as opposed to the common hand-tracking as it is not a purely manual language ASL is. The real sign language involves the use of the whole upper body and facial expression (e.g. raised eyebrows is a grammatical suffix to show that it is a question). The MediaPipe script that I wrote extracts a maximum of 1,662 specific 3D landmarks per individual frame of video:

- Pose Landmarks: 33 points which follow the shoulders, the arms and the torso to comprehend the more comprehensive physical motion and the space limits of the signer.
- Hand Landmarks: 21 points per hand, exactly reproduce the knuckles and fingertips to reproduce the finer finger-spelling and hand-shapes.
- Face Mesh Landmarks: 468 points which track the expression of the face reflecting the necessary non-manual grammatical signs and expressions.
- To achieve this dense, digital mesh successfully superimposed over the video frames was an important technical breakthrough of the project this week.

The Important Process: Data Normalization: -

The extraction of the raw coordinate data (X,Y,Z) of each landmark was a success, but in reality, the raw screen coordinates are not suitable to be used to accomplish the training of a robust neural network.

Assuming Signer A will be two feet away on the camera in the upper-left side of the image, and Signor B will be five feet away in the center of the frame, the raw spatial coordinates of the two signors will differ radically even when they are signing the same sign.

To address this variance in space, I used the second half of the week to write the preprocessing functions to normalize the data arrays. The script does not take absolute screen coordinates into the model, but rather computes relative distances and angles (e.g. making the data centra to the body of the signer himself). The model is scale and translation-invariant by normalizing the input. A user may be near a webcam or far away with the smartphone, the neural network will still identify the geometric shape of the sign in the same way.